

Rapport de Projet, Ferme autosoutenable



DEMELAS Francesco
CHAUSSARD John

Programmation orientée objet
Ingénieur informatique 1ère année

Engagement de non-plagiat

Je, soussigné(e) REBAINE Sabri, étudiant(e) en première année d'école d'ingénieur à Sup Galilée, déclare être pleinement conscient que la copie de tout ou partie d'un document, quel qu'il soit, publié sur tout support existant, y compris sur Internet, constitue une violation du droit d'auteur ainsi qu'une fraude caractérisée, tout comme l'utilisation d'outils d'Intelligence Artificielle pour générer une partie de ce rapport ou du code associé. En conséquence, je déclare que ce travail ne comporte aucun plagiat, et assure avoir cité explicitement, à chaque fois que j'en ai fait usage, toutes les sources utilisées pour le rédiger.

Fait à Sceaux, le 18/05/2025

Sabri REBAINE.

Introduction générale

Actuellement en première année d'école d'ingénieur, spécialisé en informatique au sein de l'école Sup Galilée, le contenu de ce rapport porte sur un projet réalisé dans le cadre du suivi de la matière Programmation orientée. Au cours de cette matière, nous avons été initié au langage de programmation Java qui est un langage orienté objet et centré sur le réseau, qui peut être utilisé comme une plateforme à part entière. Ce langage permet de pratiquement tout coder, que ce soit des applications mobiles ainsi que des logiciels d'entreprise en passant par les applications de big data et les technologies côté serveur. Beaucoup de jeux mobiles populaires sont également codé en java et, pour ce projet, nous avons dû réaliser d'une certaine manière un jeu pour ordinateur représentant une simulation de ferme autosuffisante.

Introduction du sujet

Lors de ce projet, nous avons construit une simulation de ferme comportant des interactions utilisateurs. Le principe du jeu est assez simple mais comporte beaucoup de spécificité.

L'utilisateur dispose d'une ferme de 6x6 cases et peut y élever des poules ainsi que des vaches, il peut également planter des tomates et du blé où construire un puit. Une même case peut être vide ou contenir :

- Un animal, un puit ou une plante individuellement
- Une plante et un animal.
- Un animal et un puit.

Les cases peuvent également contenir de l'eau et du fertilisant qui seront utiles pour les plantes.

En effet, les plantes ainsi que les animaux ont des besoins en eau et en nourriture. Lorsque leur réservoir d'eau ou de nourriture est vide de moitié, ils ont soif ou faim.

Un animal qui a faim ou soif peut manger une plante ou boire dans un puit pour restaurer complètement ses réservoirs uniquement s'il est sur la même case qu'une plante ou un puit, un animal dépose chaque jours une quantité de fertilisant sur la case où il se trouve et produit de la nourriture que l'on stock dans un entrepôt, l'animal ne peut pas consommer de fertilisant ou boire l'eau d'une case. Si le réservoir d'eau ou de nourriture d'un animal est vide, il meurt.

Une plante qui a faim ou soif peut consommer le fertilisant et l'eau présent sur la case où elle se trouve. Si la plante se trouve à proximité d'un puit, autrement dit, un puit se trouve sur une des quatre cases autour d'elle sans compter les cases diagonales, la plante peut boire indéfiniment au puit car le réservoir d'eau d'un puit est considéré comme infini. Cependant à l'inverse des animaux, les plantes n'ont pas faim car elles n'ont pas de réservoir de fertilisant.

Donc si elles ne peuvent pas consommer leur quantité de fertilisant journalière, elles sont en manque de fertilisant.

Une plante est représentée par une quantité de nourriture et croît chaque jour, si elle a soif ou est en manque de fertilisant, sa capacité de croissance diminue pour le jour en cours. Si son réservoir d'eau atteint 0, la plante meurt comme l'animal, mais une plante en manque de fertilisant ne meurt jamais. Ce n'était pas précisé dans le sujet, mais on a considéré lors du projet que si une plante pouvait consommer une certaine quantité de nourriture ou une certaine quantité d'eau mais pas assez pour remplir son réservoir, sa capacité de croissance diminue de 80% du taux de diminution de croissance comme si elle n'avait pas pu boire ou manger.

Toutes les constantes des animaux, plantes, puits et autres entités ont été placées dans un fichier `Constante.java` que nous pouvons modifier à tout moment. Les constantes de la taille de la ferme sont également dans ce fichier et peuvent être modifiées si une option d'agrandir ou de rétrécir la ferme verrait être rajoutée.

Le joueur débute avec une certaine quantité d'argent défini par la variable `argentDepart` dans la classe `Constante` et a pour but de faire prospérer sa ferme.

Enfin, des conditions spécifiques s'appliquent en fonction de la saison, il y a quatre saisons composées de 30 jours chacune et avec leur spécificité. Chaque jour les animaux se déplacent et tentent de manger ou boire s'ils ont soif ou faim et s'ils le peuvent. Lors d'une saison, le joueur ne peut pas interagir avec la simulation, en fin de saison, la nourriture produite par les animaux ainsi que la quantité de nourriture que représente les plantes est converti en argent pour le joueur, on considère que 1 calories représentent 0,75Y car ce n'était pas précisé dans le sujet. Toutes les plantes sont ensuite supprimées de la ferme et les animaux sont récupérés et peuvent être replacés pour la prochaine saison, ils disposent néanmoins d'une durée de vie, en saison, au bout de laquelle ils meurent forcément. L'utilisateur peut alors interagir et placer de nouveaux puits, plantes ou animaux ainsi qu'acheter du fertilisant pour en déposer sur les cases.

Déroulement de la simulation

Le détail du code et des méthodes est donné en commentaire du code et est normalement présent dans la JavaDoc mais voici son fonctionnement global.

Nous avons créé une classe `Entite` afin de regrouper les comportements similaires que peuvent avoir les plantes et les animaux, comme le fait qu'ils ont tous deux un prix et disposent d'un réservoir d'eau, les animaux ont une capacité de déplacement de 1 case par jour et les plantes de 0. On peut cependant imaginer une évolution du jeu où les plantes pourraient être déplacées par un fermier de manière aléatoire comme les animaux. On a donc

défini une méthode abstraite dans la classe *Entite* *gererDeplacement(Ferme f)* que nous devrons implémenter dans les classes filles de *Entite*, donc *Animal* et *Plante*.

De même pour le nombre de déplacement, on pourrait imaginer vouloir augmenter la capacité de déplacement d'un animal à plus de 1 cases par jours. Si l'animal a plusieurs déplacements, on part du principe qu'en fonction de s'il a faim ou soif, il va chercher à boire ou manger et qu'une fois son objectif atteint, il va arrêter de se déplacer. De même pour les plantes, à l'exception qu'une plante gelée ne peut pas se déplacer car, en hiver, les tomates gèlent, elles ne grandissent plus et ne peuvent plus non plus être mangées par un animal jusqu'à la fin de la saison.

Cette méthode est appelée depuis la classe *Simulation* dans la méthode

gererAnimal(Animal a), elle-même appelée par *gererAnimaux()* qui parcourt tous les animaux de notre ferme et gère leur comportement individuellement en fonction de leurs besoins ainsi que des saisons. À la fin du jour, on essaye de récupérer les valeurs de consommation journalière de nourriture et d'eau à l'aide d'une boucle try-catch. La méthode pour récupérer ces quantités est définie dans la classe *Reservoir*, elle lève deux types d'exception :

- *CapaciteeDepasseeException*, si le réservoir est vide
- *NiveauInsuffisantException*, si le réservoir n'est pas vide mais ne peut pas délivrer la quantité demandée.

Donc, pour un animal, si on attrape une de ses deux exceptions, cela signifie que le niveau du réservoir est nul où sera nul après avoir retiré la quantité demandée, l'animal meurt donc et on appelle la méthode *f.suppAnimalTemp(a)*, définie dans la classe *Ferme*. Cette méthode permet d'ajouter notre animal à une liste indiquant les animaux à supprimer après le parcours des tous nos animaux par la boucle *foreach* les parcourant.

En revanche, pour une plante, si on attrape l'exception *CapaciteeDepasseeException*, lorsqu'on essaye de récupérer la quantité d'eau qu'elle consomme chaque jour, la plante meurt comme pour l'animal. Cependant, si on attrape l'exception *NiveauInsuffisantException*, alors on diminue sa croissance de 80% de sa diminution de croissance ordinaire en cas de soif, de même lorsque l'on essaye de récupérer le fertilisant présent sur la case.

Après des tests sur les animaux pour vérifier qu'ils mourraient correctement sans eau ni nourriture, en ne plaçant qu'une poule sur la grille et avec le code suivant dans la méthode *gererAnimal* :

```

a.consommerEau(consoEau);

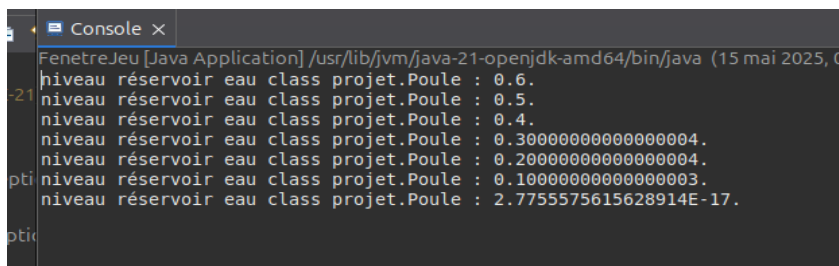
System.out.println("niveau réservoir eau "+a.getClass()+" :
"+a.getNiveauEau()+".");

a.consommerNour(consoNour);

System.out.println("niveau réservoir nourriture "+a.getClass()+" :
"+a.getNiveauNour()+".");

```

On a pu observer ce résultat :



```

FenetreJeu [Java Application] /usr/lib/jvm/java-21-openjdk-amd64/bin/java (15 mai 2025, 0
niveau réservoir eau class projet.Poule : 0.6.
niveau réservoir eau class projet.Poule : 0.5.
niveau réservoir eau class projet.Poule : 0.4.
niveau réservoir eau class projet.Poule : 0.300000000000000004.
niveau réservoir eau class projet.Poule : 0.200000000000000004.
niveau réservoir eau class projet.Poule : 0.100000000000000003.
niveau réservoir eau class projet.Poule : 2.775575615628914E-17.

```

On remarque que quelque chose ne se produit pas comme prévu. Initialement, une poule dispose de 0.7L d'eau et elle consomme 0.1L d'eau par jours, le niveau de son réservoir devrait donc atteindre 0 au bout du 7ème jour. Cependant, on remarque qu'au bout du 7ème jour, il lui reste encore environ 2.7×10^{-17} L d'eau et elle ne meurt donc qu'au bout du 8ème jours.

Ce problème vient du fait que pour calculer une valeur décimale, un ordinateur réalise un arrondi, donc pour décrémenter de très petits nombres flottants, cela peut créer un léger décalage, ce que l'on constate au bout du 4ème jours où le niveau du réservoir de la poule est des de 3.000...04 au lieu de 3.0. Cela provoque qu'il lui reste une petite quantité d'eau dans son réservoir et la poule ne meurt pas au bout du 7ème jours mais au bout du 8ème.

Afin de rester réaliste avec les demandes de l'énoncé, la poule doit cependant mourir au bout du 7ème jours, on aimerait donc que le niveau du réservoir soit assimilé à 0 s'il contient une valeur très petite, pour cela on a modifié la méthode *recuperer* de la classe *Reservoir* ainsi :

Au lieu de vérifier quand le niveau du réservoir est inférieur ou égal à 0, on effectue cette vérification pour vérifier si le reservoir est vide :

if (*niveau* <= 0 + *marge*), avec la marge définie ainsi :

double *marge* = 0.000001;

On définit une marge très petite afin d'être sûr de ne pas influencer le résultat de cette méthode autre que si le niveau du réservoir est très proche de 0 et on compare le niveau à 0+marge. Avec cette méthode, 2.7×10^{-17} est inférieur à 0+marge et la méthode lance

bien l'exception *CapaciteDepasseeException* si le réservoir est considéré comme vide et notre poule meure bien au bout du 7ème jour.

A la fin d'une saison, on vide d'abord la grille de tous les animaux et plantes encore présent dans la méthode *viderGrille()* et on récupère l'argent correspondant aux productions des animaux pendant la saison ainsi que la quantité de nourriture que représente chaque plante multipliée par 0.75, le pourcentage de conversion entre calories et Y. grâce à la méthode *recolter()* appelée au début de *viderGrille()* et on passe en phase de placement s'il nous reste des animaux en vie. Au début de la saison suivante, les animaux ont leur réservoirs remplis et, si c'était des nouveaux nés ils deviennent adultes, car au Printemps, les animaux de la même espèce et côté à côté peuvent se reproduire selon une certaine probabilité, les animaux nouveaux nés ont leur constante divisée par 2 à l'exception de leur capacité de déplacement ainsi que leur durée de vie, de plus, ils ne produisent pas de fertilisant.

Enfin, on met en pause la simulation pour laisser l'utilisateur interagir et on réinitialise l'entrepôt à 0, le jour à 1 et on lance prépare la prochaine saison en incrémentant de 1 modulo 4 l'entier *saison*, puis en réalisant un *switch-case* sur sa valeur : s'il vaut 0, on est en hiver, 1 au printemps, 2 en été et 3 en automne, sachant que sa valeur initiale est choisie aléatoirement entre 0 et 3 inclus. Lorsque l'utilisateur appuiera sur démarrer, on affecte *false* au booleen *enPause*, et on démarre la simulation de la saison suivante.

Implémentation de l'interface graphique

L'implémentation de l'interface graphique repose sur les classes *Affichage* et *FenetreJeu*. Initialement, nous ne disposons pas de ces classes et l'affichage se faisait sur la console à l'aide de commande *System.out.println()* pour afficher du texte ainsi que la grille et de *Scan.nextInt()* pour demander des valeurs à l'utilisateur, par exemple s'il rentrait 1, cela correspondait à l'achat d'une poule.

Une fois le développement du projet presque achevé, je me suis aidé d'outils d'intelligence artificielle afin de réaliser cette interface graphique. J'ai principalement utilisé Chatgpt ainsi que ClaudeAi pour m'aider. ClaudeAi, apporte généralement de meilleures réponses mais ne peut pas contenir de fichiers trop lourds où de texte trop long. J'ai donc fourni la plupart de mon code à ChatGpt en créant et lui fournissant un merge file .txt ainsi que le sujet du devoir, en lui demandant comment je pourrai relier ce que j'ai réalisé à une interface graphique et lui expliquant ce que je voulais. Pour des petits problèmes avec un affichage, je me tournais plus vers ClaudeAi si ChatGpt ne parvenait pas à m'expliquer clairement comment procéder. Les premiers tests n'étaient pas très concluants mais en continuant de lui expliquer, je devais apporter des modifications à certaines parties du code, cela m'a permis de comprendre dans la globalité son fonctionnement et d'avoir un squelette de construction d'interface graphique, j'ai donc par la suite pu faire quelques modifications en fonction de comment je voulais avoir mon, affichage final, par exemple demander le nom à l'utilisateur avant de démarrer, où

encore afficher des petites images sur les cases contenant de l'eau et du fertilisants. Les IA ainsi que quelques recherches m'ont donc grandement permis de réaliser cette interface graphique et d'avoir une vision de comment cela pouvait fonctionner, ce qui m'était encore inconnu auparavant.

Conclusion

Pour conclure, je dirai que ce projet m'a permis de grandement fortifier mes connaissances en java et en programmation orientée objet, avant cela les notions d'objets, d'héritages et surtout d'exceptions étaient encore floues mais j'ai maintenant été forcé de pleinement les utiliser et donc de beaucoup mieux les appréhender. J'ai beaucoup apprécié réaliser cette simulation car cela m'a beaucoup appris et il était satisfaisant de faire fonctionner la ferme et de réaliser ma toute première interface graphique. L'aspect d'interfaces graphique comme le déclenchement d'un timer et la mise à jour de la grille tous les jours avec ½ secondes de délais entre chaque jour m'était complètement inconnue et j'ai pu les implémenter et modifier à ma guise grâce aux squelettes fournis par les intelligences artificielles afin d'obtenir un rendu qui me convient.